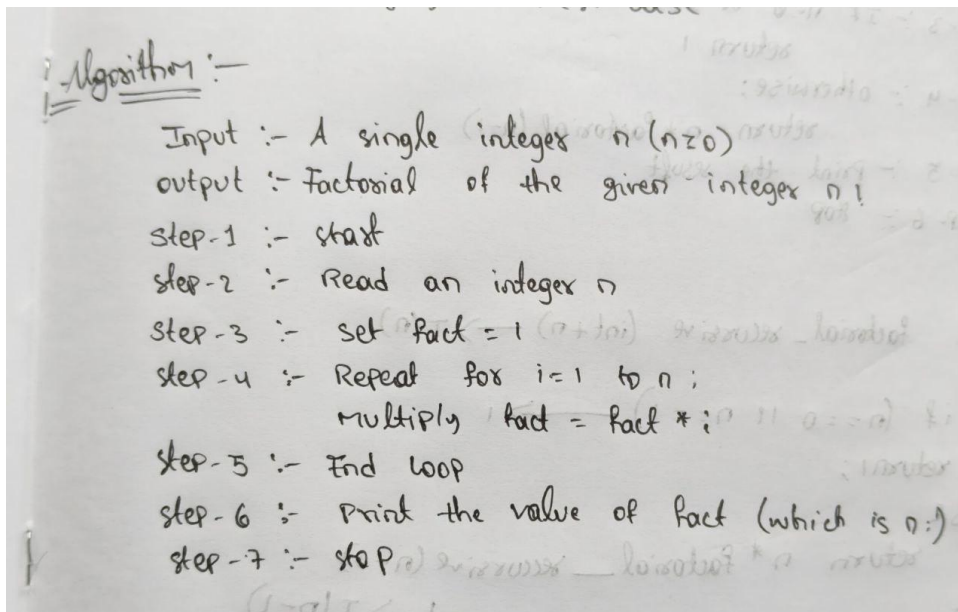


PRACTICAL-04

AIM: Implementation and Time analysis of factorial program using iterative and recursive method.

FACTORIAL BY ITERATIVE

ALGORITHM:



Algorithm :-

- Input :- A single integer n ($n \geq 0$)
- output :- Factorial of the given integer $n!$
- step-1 :- start
- step-2 :- Read an integer n
- step-3 :- set $fact = 1$ ($n \geq 0$)
- step-4 :- Repeat for $i = 1$ to n ;
 multiply $fact = fact * i$;
- step-5 :- End loop
- step-6 :- Print the value of fact (which is $n!$)
- step-7 :- stop

PROGRAM:

```
#include <iostream>

using namespace std;

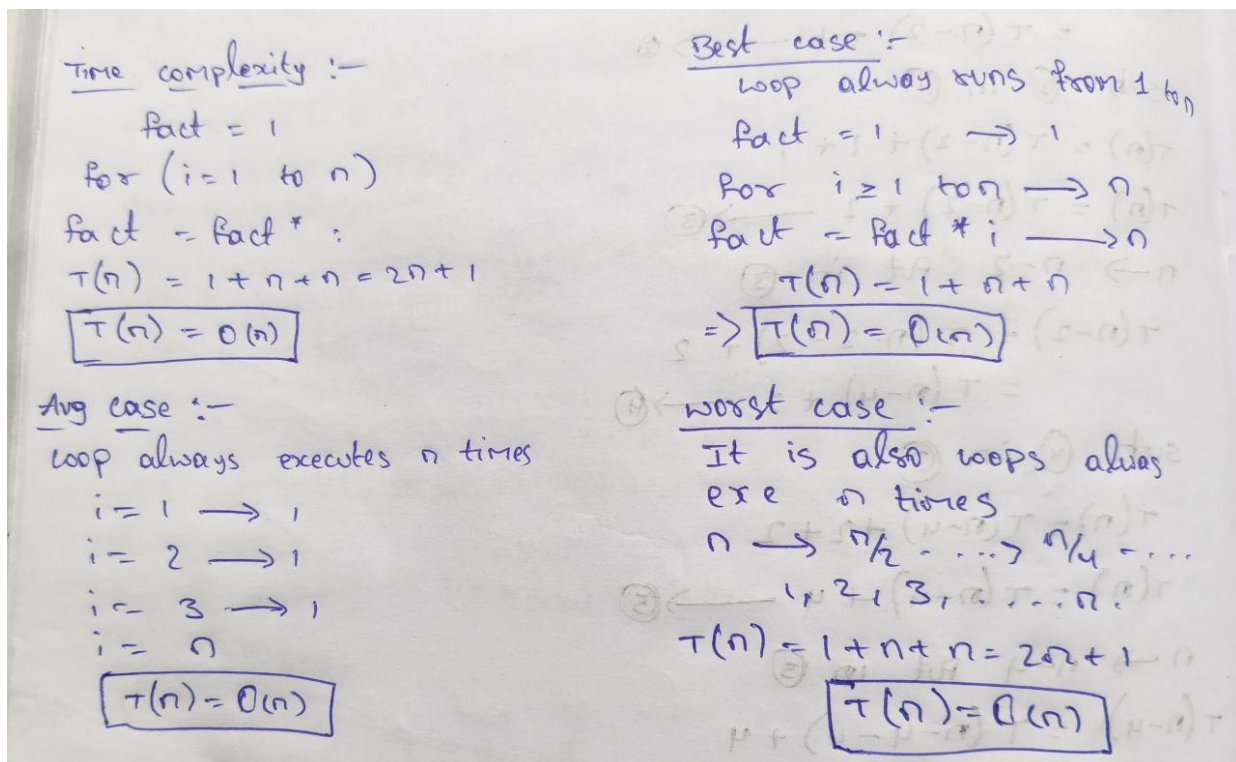
int factorial_iterative(int n) {
    int fact = 1;
    for (int i = 1; i <= n; i++) {
        fact = fact * i;
    }
    return fact;
}
```

```
int main() {  
    int n;  
    cout << "Enter a number: ";  
    cin >> n;  
    cout << "Factorial (Iterative): " << factorial_iterative(n) << endl;  
    return 0;  
}
```

OUTPUT:

```
Enter a number: 5  
Factorial (Iterative): 120
```

TIME COMPLEXITY:



Time complexity :-
Fact = 1
for (i = 1 to n)
fact = fact * i
 $T(n) = 1 + n + n = 2n + 1$
 $T(n) = O(n)$

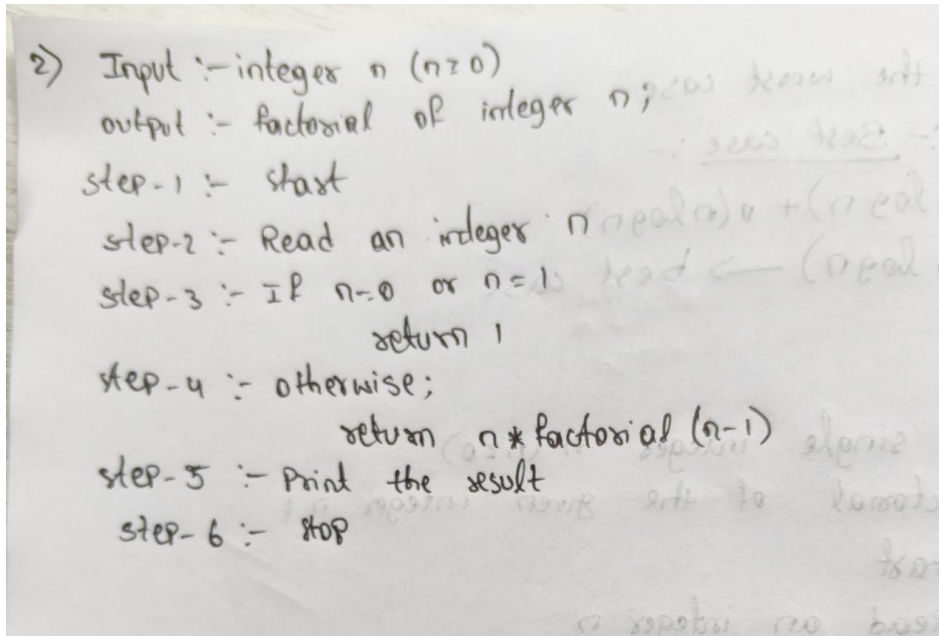
Avg case :-
loop always executes n times
i = 1 $\rightarrow$ 1
i = 2 $\rightarrow$ 1
i = 3 $\rightarrow$ 1
i = n
 $T(n) = O(n)$

Best case :-
loop always runs from 1 to n
Fact = 1 $\rightarrow$ 1
for i = 1 to n $\rightarrow$ n
fact = fact * i $\rightarrow$ n
 $T(n) = 1 + n + n$
 $\Rightarrow T(n) = O(n)$

worst case :-
It is also loops always
exe n times
n $\rightarrow$ n/2 $\dots$ n/4 $\dots$
n/2, 3, $\dots$ n
 $T(n) = 1 + n + n = 2n + 1$
 $T(n) = O(n)$

FACTORIAL BY RECURSIVE

ALGORITHM:



2) Input :- integer n ($n \geq 0$)
output :- factorial of integer n
step-1 :- start
step-2 :- Read an integer n
step-3 :- if $n=0$ or $n=1$ then return 1
step-4 :- otherwise;
return $n * \text{factorial}(n-1)$
step-5 :- Print the result
step-6 :- stop

PROGRAM:

```
#include <iostream>

using namespace std;

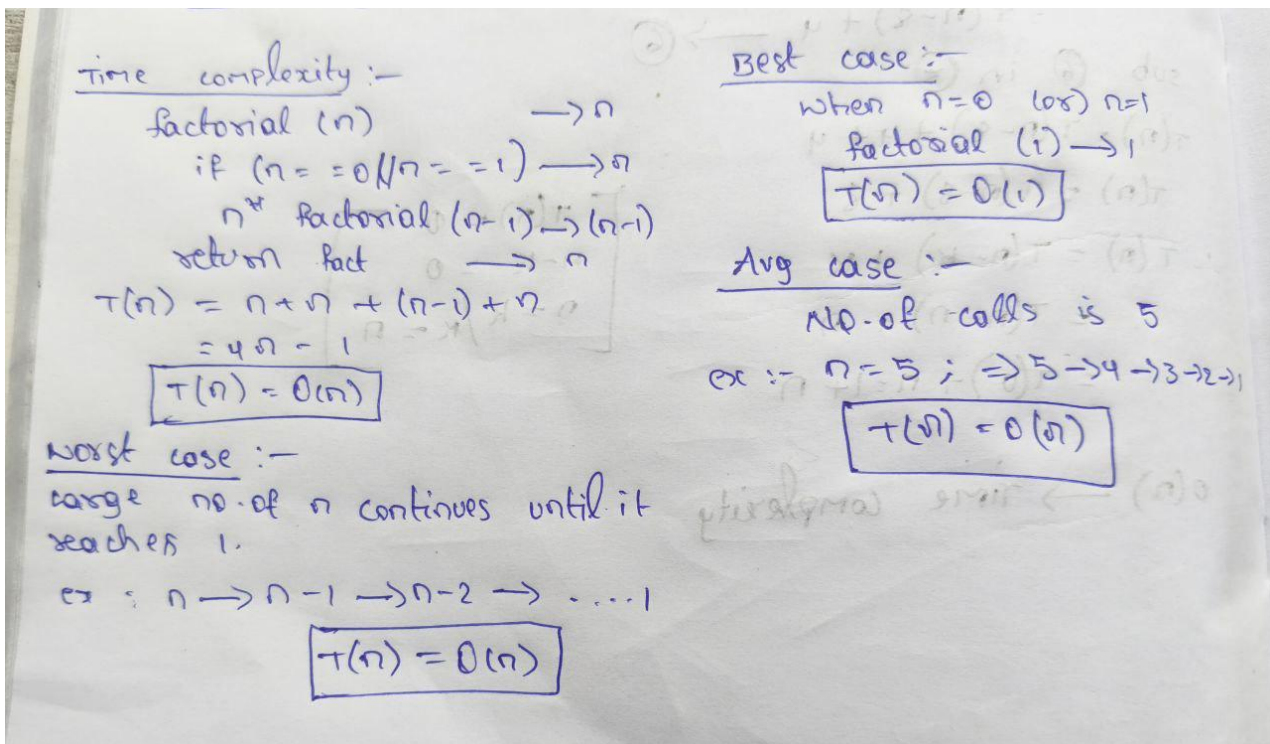
int factorial_recursive(int n) {
    if (n == 0 || n == 1)
        return 1;
    else
        return n * factorial_recursive(n - 1);
}

int main() {
    int n;
    cout << "Enter a number: ";
    cin >> n;
    cout << "Factorial (Recursive): " << factorial_recursive(n) << endl;
    return 0;
}
```

OUTPUT:

```
Enter a number: 5
Factorial (Recursive): 120
```

TIME COMPLEXITY:



Time complexity :-

factorial (n) $\rightarrow n$
if (n == 0 || n == 1) $\rightarrow 1$
nth factorial: (n-1) $\rightarrow (n-1)$
return fact $\rightarrow n$
 $T(n) = n + n + (n-1) + n$
 $= 4n - 1$
 $T(n) = O(n)$

Worst case :-
large no. of n continues until it reaches 1.
ex : $n \rightarrow n-1 \rightarrow n-2 \rightarrow \dots \rightarrow 1$
 $T(n) = O(n)$

Best case :-
when $n=0$ (or) $n=1$
factorial (i) $\rightarrow 1$
 $T(n) = O(1)$

Avg case :-
No. of calls is 5
ex :- $n=5 ; \Rightarrow 5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1$
 $T(n) = O(n)$

CONCLUSION:

The factorial program demonstrates both iterative and recursive approaches. Iterative factorial runs in linear time $O(n)$ with constant space, making it efficient. Recursive factorial also has $O(n)$ time but requires $O(n)$ stack space due to repeated function calls. Thus, iteration is more memory-efficient, while recursion is conceptually elegant.